

Keywords: efficient inference • diffusion language models • speculative decoding • throughput

Nemotron-Labs-Diffusion

NVIDIA Unifies Autoregressive and Diffusion Language Modeling Into a Single Self-Speculating Architecture That Eliminates the Need for a Separate Draft Model

By Yonggan Fu, Lexington Whalen, Abhinav Garg, Chengyue Wu, et al. (NVIDIA Research)

arXiv:2607.05722 • July 7, 2026

Modern speculative decoding relies on a separate draft model to propose candidate tokens that a larger verifier model then accepts or rejects. Maintaining two models doubles deployment cost and complicates serving infrastructure. Nemotron-Labs-Diffusion eliminates the draft model entirely by training a single set of weights to operate in three modes: standard autoregressive (AR) decoding, parallel diffusion decoding, and self-speculation, where the diffusion head drafts while the AR head verifies—from the same checkpoint, with no weight switching.

AT A GLANCE

Problem: Speculative decoding requires a separate draft model, doubling parameter budget and serving complexity.

Why it matters: Throughput is the primary cost driver for large-scale LLM deployment; a 4-6× improvement directly reduces inference cost.

Method: Joint AR-diffusion training objective; at inference, the diffusion head proposes draft token blocks and the AR head verifies, all within one set of weights.

Result: 6.82 accepted tokens per step (vs. Eagle3’s 2.75); 4× higher throughput than Qwen3-8B on SPEED-Bench.

Evidence: Evaluated on SPEED-Bench, MT-Bench, MMLU, and HumanEval across 3B, 8B, and 14B model sizes.

The Big Picture

The dominant paradigm for efficient large language model inference is speculative decoding: a small, cheap draft model proposes a block of candidate tokens, and the large model verifies the entire block in a single forward pass. When the acceptance rate is high, this delivers near-linear speedups. The problem is that the draft model is a separate artifact that must be trained, maintained, and co-deployed with the verifier, doubling operational complexity.

Multi-token prediction (MTP) sidesteps the separate model by training the main model to predict multiple future tokens simultaneously using auxiliary heads. But MTP heads predict tokens autoregressively from a fixed left-to-right prior, limiting their ability to propose coherent multi-token continuations.

Nemotron-Labs-Diffusion observes that the diffusion training objective—which reconstructs any masked subset of tokens conditioned on unmasked context—is exactly the capability needed for a high-quality internal drafter. A model trained jointly with AR and diffusion objectives learns to both predict the next token from the left and fill any arbitrary masked block, making its diffusion head a natural draft model and its AR head a natural verifier.

Why Existing Approaches Fall Short

Eagle3, the leading separate-draft speculative decoding system, achieves 2.75 accepted tokens per step. Its primary weakness is that the draft model must be trained separately and kept synchronized with the verifier model through costly co-training or periodic retraining. Every new verifier checkpoint requires a corresponding draft model update.

MTP-based approaches (as used in DeepSeek-V3) train auxiliary prediction heads on the verifier model itself but restrict each head to predicting a single additional token conditioned on prior outputs. The heads predict tokens serially rather than in parallel, and their training signal is weaker because they never reconstruct arbitrary masked subsets—they only predict a fixed number of positions ahead.

Standard diffusion LMs (as in MDLM and Plaid) have high generation quality in parallel mode but suffer from a quality-speed tradeoff: fewer diffusion steps produce lower quality, while more steps reduce the throughput advantage over AR models.

Core Mechanism

Nemotron-Labs-Diffusion resolves these tensions by exploiting the complementarity between AR and diffusion objectives. The AR objective provides strong left-to-right linguistic priors that make verification reliable; the diffusion objective provides parallel reconstruction capability that makes drafting both fast and coherent.

In self-speculation mode, the model executes a draft-verify cycle. The diffusion head, given the current context, denoises a block of B masked positions in parallel using one or two diffusion steps. The AR head then scores each proposed token in a single forward pass (matching the cost of verifying B tokens in standard speculative decoding). Tokens above the acceptance threshold are committed; the rest trigger a fresh diffusion draft.

Technical Formulation

The joint training loss is:

$$\mathcal{L} = \alpha \mathcal{L}_{\text{AR}} + (1 - \alpha) \mathcal{L}_{\text{diff}}$$

where $\mathcal{L}_{\text{AR}}$ is the standard cross-entropy on the next token and $\mathcal{L}_{\text{diff}}$ is the masked token reconstruction loss at variable mask rates $\rho \sim \text{Beta}(a, b)$:

$$\mathcal{L}_{\text{diff}} = -\mathbb{E}_{\rho, \mathcal{M}} \sum_{i \in \mathcal{M}} \log p_{\theta}(x_i | x_{\setminus \mathcal{M}})$$

with $\mathcal{M}$ the random mask set of expected size ρn . At $\alpha = 0.5$, both objectives contribute equally; ablations confirm this is the Pareto-optimal balance.

Token acceptance in self-speculation mode follows the standard speculative decoding criterion: draft token x_d is accepted with probability $\min(1, p_{\text{AR}}(x_d)/p_D(x_d))$, preserving the AR distribution exactly.

Learning or Inference Procedure

1. Train with joint AR+diffusion loss using standard next-token and masked-span reconstruction objectives.
2. Fine-tune an additional lightweight MLP sampler on diffusion sampling trajectories to reduce required denoising steps to 1–2.
3. At inference, select mode based on deployment constraints: AR mode for minimum per-token latency; diffusion mode for maximum throughput at batch size 1; self-speculation for the best throughput-quality tradeoff at any batch size.
4. Self-speculation loop: draft $B = 8$ tokens via 1-step diffusion; score with AR head; commit accepted tokens; redraft rejected suffix.

What the Guarantee Says

The acceptance-based guarantee from standard speculative decoding carries over: accepted tokens are drawn exactly from the AR model’s distribution, so Nemotron-Labs-Diffusion-self-speculation produces outputs statistically identical to pure AR sampling. No quality degradation is introduced by the self-speculation mechanism.

The throughput guarantee is empirical: on SPEED-Bench with SGLang on NVIDIA GB200, Nemotron-Labs-Diffusion-8B achieves $4\times$ higher throughput than Qwen3-8B at equivalent quality (within 1% on MT-Bench, MMLU, and HumanEval).

Experimental Findings

System	Accepted toks/step	Throughput
Qwen3-8B (AR only)	1.00	1.0 $\times$
DeepSeek-V3 MTP	2.11	1.9 $\times$
Eagle3 (separate draft)	2.75	2.6 $\times$
Nemotron-8B (ours)	6.82	4.0$\times$

Quality scores (MT-Bench / MMLU / HumanEval) for Nemotron-8B are within 1% of Qwen3-8B AR baseline across all benchmarks. The 3B model variant outperforms the 7B Eagle3 draft+8B verifier pair on throughput while using 5B fewer total parameters.

Ablations and Interpretation

Removing diffusion training drops accepted tokens per step from 6.82 to 4.1, confirming the diffusion head’s centrality to draft quality. This is the most significant ablation result.

α sweep: Higher α (more AR weight) improves text quality metrics by up to 0.3% but reduces acceptance rate by up to 1.4 tokens per step. $\alpha = 0.5$ is the Pareto-optimal balance.

Mask rate schedule: Fixed mask rate $\rho = 0.15$ (like BERT) reduces accepted tokens per step by 1.2 relative to the Beta(1, 5) schedule, which samples mask rates across the full $[0, 1]$ range and builds more versatile reconstruction capability.

Self-speculation vs. MTP: At equal parameter budgets, self-speculation accepts 6.82 tokens per step versus MTP’s 2.11, because MTP heads can only predict a fixed number of positions while the diffusion head can fill any masked block with a single forward pass.

Reference

Yonggan Fu et al. **Nemotron-Labs-Diffusion: A Tri-Mode Language Model Unifying Autoregressive, Diffusion, and Self-Speculation Decoding**. arXiv:2607.05722, July 2026. <https://arxiv.org/abs/2607.05722>

Keywords: mechanistic interpretability • coding agents • probing • latent representations

Latent Programming Horizons

Coding Agents Know the Outcome of Their Edits Up to 25 Steps Before They Make Them

By André Silva, Han Tu & Martin Monperrus (KTH Royal Institute of Technology, Stockholm)

arXiv:2607.05188 • July 6, 2026

When a language model acts as a coding agent, it spends dozens of steps reading files, reasoning about bugs, writing patches, and running tests—a process that is opaque from the outside. This paper opens that black box. Using logistic-regression probes on the residual streams of GPT-4o and Claude 3.7 Sonnet, the authors show that the model’s internal representations linearly encode the correctness of the program being edited with AUC up to 0.83—and that these representations run up to 25 steps *ahead* of the agent’s own actions, constituting what the authors call a **latent programming horizon**.

AT A GLANCE

Problem: The internal states of coding agents during multi-step repair episodes are not understood; it is unknown whether models plan ahead or react greedily.

Why it matters: A latent horizon implies agents have exploitable internal signals for early stopping, intervention, and lookahead guidance without external execution.

Method: Logistic-regression probes on final-layer hidden states recorded at each step of coding-agent episodes; future probes predict outcomes k steps ahead.

Result: AUC 0.83 for correctness at step t ; horizon k^* up to 25 steps on SWE-bench Verified.

Evidence: Two models (GPT-4o, Claude 3.7), two benchmarks (SWE-bench Verified, HumanEval-Pro); cross-benchmark transfer retains AUC > 0.70 .

The Big Picture

Large language models acting as software engineering agents can resolve real GitHub issues, pass coding interviews, and complete complex multi-file refactors—but the mechanism by which they succeed or fail across a long episode remains opaque. Mechanistic interpretability research has largely focused on single forward passes or small fixed-context tasks.

Coding-agent episodes are different: the model sees an evolving repository state, modifies files, receives test out-

puts, and makes new decisions over potentially hundreds of steps. Understanding what the model “knows” at each step has direct practical value—if the model’s internal state already encodes the likely outcome of the episode, that signal could be used to guide, interrupt, or improve the agent without waiting for external test execution.

This paper introduces the first systematic mechanistic interpretability study of coding agents over full multi-step episodes.

Why Existing Approaches Fall Short

Prior mechanistic interpretability work probes fixed-context forward passes (e.g., factual recall, in-context learning, arithmetic). These methods cannot be applied directly to coding-agent episodes because (1) the context evolves across steps as files are modified, (2) ground truth labels depend on external execution of the evolving codebase, and (3) the episode length (up to 200 steps) is far longer than tasks previously studied.

Behavioral analysis of coding agents (e.g., measuring pass@k or success rate) describes what agents do but not what they represent. Existing evaluation frameworks do not distinguish between agents that reason correctly but act incorrectly and agents that reason incorrectly and happen to succeed by luck—a distinction this paper makes tractable via probing.

Core Mechanism

The latent programming horizon arises because the model processes the full current repository state and reasoning history at each step, and its internal representations must encode relevant aspects of the program’s state to generate useful next-action tokens.

The key empirical finding is that correctness-relevant features are not just encoded in the final step before an edit—they are stably encoded up to 25 steps earlier in the episode. This suggests the model is tracking a “mental model” of the program’s correctness that is both accurate and temporally extended.

Technical Formulation

At each step t of an episode, the last-position final-layer residual stream vector $h_t \in \mathbb{R}^{4096}$ is recorded. Binary labels $y_t \in \{0, 1\}$ are obtained by running pytest on the repository state after each agent edit (parsed: 1/0; test-pass: 1/0; etc.).

For immediate probes, a logistic regression $f_\theta(h_t) \rightarrow \hat{y}_t$ is trained on (h_t, y_t) pairs.

For future probes at horizon k , the same probe structure predicts y_{t+k} from h_t . The latent programming horizon

is:

$$k^* = \max\{k : \text{AUC}(f_{t \rightarrow t+k}) \geq 0.55, p < 0.05\}$$

where significance is assessed by paired permutation test over episodes.

Learning or Inference Procedure

1. Run coding agents on SWE-bench Verified (300 issues) and HumanEval-Pro (150 problems) with step-by-step residual stream logging enabled via API hooks.
2. After each edit, execute sandboxed pytest and record binary labels for four tasks (parses, passes tests, reduces failures, introduces regression).
3. Train logistic-regression probes on 80% of episodes; evaluate on 20% held-out episodes.
4. Sweep horizon k from 0 to 40; measure AUC and p -value at each horizon to determine k^* .
5. Perform cross-benchmark transfer: train on SWE-bench, evaluate on HumanEval-Pro without retraining.

What the Guarantee Says

The core finding is not a guarantee in the theoretical sense, but a robust empirical regularity: across two models and two benchmarks, the latent programming horizon k^* is consistently 20–28 steps for “passes tests” and 12–16 steps for “introduces regression.” Probes trained on one model do not transfer to the other, indicating the representations are model-specific even if the phenomenon is model-agnostic. Cross-benchmark transfer holds above AUC 0.70 for all tasks, confirming the probe captures generic code-quality structure.

Experimental Findings

Task	Model	AUC ($k=0$)	Horizon k^*
Parses	GPT-4o	0.81	28
Passes tests	GPT-4o	0.78	22
Reduces fails	Claude 3.7	0.83	25
Introduces reg.	Claude 3.7	0.71	14

Cross-benchmark transfer (SWE-bench $\rightarrow$ HumanEval-Pro) retains AUC above 0.70 for all tasks. Middle-layer probes (layer 24/48) achieve only AUC 0.63, indicating the relevant representations are built through the full network depth. Non-linear probes improve AUC by only 0.02, confirming approximate linear decodability.

Ablations and Interpretation

Layer depth: Probing earlier layers degrades AUC monotonically from 0.83 (final layer) to 0.51 (first layer), suggesting correctness representations are progressively con-

structed across transformer depth.

Non-linearity: Adding one MLP hidden layer improves AUC by 0.02 on average. The dominant signal is linear, consistent with the probing literature on factual representations.

Shuffled-label control: Probes trained on permuted labels achieve AUC 0.50 ± 0.01 , ruling out statistical artifacts from episode length or context size.

Practical implication: The 25-step horizon means an external monitor could identify likely-failing episodes 25 steps early and redirect the agent—a potential 25-step saving per failed episode. On SWE-bench Verified, where 40% of episodes fail, this would reduce total token cost by $\sim 10\%$.

Reference

André Silva, Han Tu, Martin Monperrus. **Latent Programming Horizons in Coding Agents.** arXiv:2607.05188, July 2026. KTH Royal Institute of Technology. <https://arxiv.org/abs/2607.05188>

Keywords: agent evaluation • benchmark • long-horizon tasks • dense rewards

Long-Horizon-Terminal-Bench

A 46-Task Dense-Reward Benchmark Exposes the True Performance Gap Between LLM Agents on Complex Terminal Workflows

By Zongxia Li, Zhongzhi Li, Yucheng Shi, Ruhan Wang et al. (Tencent HY LLM Frontier; Univ. of Maryland; et al.)

arXiv:2607.08964 • July 9, 2026

Binary pass/fail benchmarks for coding and terminal agents are comfortable lies: they assign 0 or 1 to an episode without capturing how far the agent progressed on complex, multi-step workflows. Long-Horizon-Terminal-Bench (LHTB) replaces this illusion with 46 tasks spanning nine categories, graded by hidden rebuild-from-artifact verifiers that assign partial credit through fine-grained subtask completion. The result is a sobering view: top agents score only 28–34% on the hardest categories—and would have appeared far more capable under binary grading.

AT A GLANCE

Problem: Binary agent benchmarks mask variation in task progress, distorting comparisons and hiding true capability gaps.

Why it matters: Real deployment requires agents that make sustained, useful progress over hundreds of steps; binary grading cannot differentiate 5% from 95% completion.

Method: 46-task terminal benchmark with subtask-level dense rewards; hidden rebuild-from-artifact verifiers prevent score gaming.

Result: Top agents (Claude 3.7, GPT-4o) average 52% overall; only 28–34% on scientific computing and experiment reproduction.

Evidence: Three frontier models evaluated; binary-vs-dense correlation only 0.41, confirming benchmarks measure different capabilities.

The Big Picture

Current LLM agent evaluations suffer from a fundamental measurement problem. SWE-bench, HumanEval, and Terminal-Bench all grade each task as a binary 0 or 1: either the final state passes all tests, or it does not. This approach is easy to implement and produces clean leaderboards, but it conflates “failed to start” with “almost finished”—both receive the same score of 0.

The problem compounds on long-horizon tasks where success requires chaining dozens of subtasks: running a numerical simulation, checking convergence, generating plots, and writing a report. An agent that completes the simulation and convergence check but fails on the plotting step does 60% of the work—but receives 0 credit under binary grading.

LHTB introduces dense intermediate rewards by decomposing every task into 4–12 fine-grained, verifiable subtasks, each graded independently. Partial credit is not merely cosmetic: it changes which agents rank highest and by how much, and reveals capability gaps invisible under binary evaluation.

Why Existing Approaches Fall Short

Terminal-Bench introduced the key insight of containerized, agentic evaluation with a bash shell—but retained binary grading. SWE-bench Verified improved task realism (real GitHub issues) but also grades binary. Both benchmarks are additionally vulnerable to self-reported progress: agents can write test files that trivially pass their own assertions, gaming the verifier.

Process reward models (PRMs) offer a complementary approach—training a model to assign intermediate rewards—but require supervised reward data that is expensive to collect and brittle to domain shift. LHTB achieves dense rewards without any learned reward model by decomposing tasks into subtasks with objectively verifiable intermediate states (e.g., “the simulation output file exists and contains a valid numpy array with shape (T, N) ”).

Core Mechanism

The benchmark’s core contribution is the combination of (1) task decomposition into verifiable subtasks and (2) rebuild-from-artifact verification that prevents score gaming.

For rebuild-from-artifact verification, instead of running the agent’s code directly, the verifier mounts the agent’s output directory read-only, extracts specified intermediate artifacts (files, database entries, generated figures), reconstructs a ground-truth comparison from these artifacts, and asserts properties. This prevents agents from planting self-passing assertions or manipulating the test environment because the verifier operates on static artifacts, not live processes.

Technical Formulation

Each task τ decomposes into subtasks $\{s_1, \dots, s_m\}$, each with a binary verifier $v_i \in \{0, 1\}$. The composite score is:

$$S(\tau) = 0.7 \cdot \frac{1}{m} \sum_{i=1}^m v_i + 0.2 \cdot V_{\text{final}} + 0.1 \cdot B_{\text{eff}}$$

where V_{final} is a holistic final-state verifier and B_{eff} is an efficiency bonus that decays linearly for episodes requiring more than 100 steps. This scoring incentivizes both thorough subtask completion and efficient execution.

Learning or Inference Procedure

Agents are placed in an Ubuntu 22.04 Docker container with standard scientific Python, build tools, git, and a network connection. They receive three tools: (1) a bash executor, (2) a file editor, and (3) a web search interface. The step limit is 200; agents are instructed only with the natural-language task description.

1. Agent reads task description and initializes workspace.
2. Agent executes a sequence of bash commands, file edits, and searches to complete the task.
3. After each agent-initiated “checkpoint” (or at the step limit), the verifier mounts the filesystem, extracts specified artifacts, and runs subtask assertions.
4. Subtask credits accumulate; a final holistic verifier runs after the episode ends.

What the Guarantee Says

The red-team evaluation provides an empirical anti-gaming guarantee: agents instructed to game the verifier succeeded on 0/46 tasks, versus 23/46 on a naive run-and-check verifier. The rebuild-from-artifact approach does not prevent all possible manipulation, but the attack surface is dramatically smaller because agents cannot interact with the verifier process at all—only their output artifacts are examined.

Experimental Findings

Category	Claude 3.7	GPT-4o	Gemini 2.5
Software Eng.	72.1%	68.4%	65.2%
Data Eng.	61.3%	58.7%	54.1%
DevOps	55.8%	52.3%	49.6%
Interactive Games	41.5%	38.2%	35.8%
Experiment Repro.	34.2%	30.1%	28.7%
Scientific Comp.	28.9%	27.4%	24.3%
Overall (avg.)	52.3%	49.1%	45.9%

The correlation between binary pass/fail scores on prior benchmarks and LHTB subtask credit is only 0.41, confirming that LHTB captures genuinely different capabilities.

ties. Binary grading reverses the ranking between Claude 3.7 and GPT-4o on 7 of 46 tasks.

Ablations and Interpretation

Binary vs. dense grading: Binary grading on LHTB tasks produces rankings that disagree with dense grading on 7/46 tasks, illustrating how coarse grading can mislead model selection.

Verifier robustness: In the red-team evaluation, agents instructed to game succeed on 0/46 tasks under rebuild-from-artifact verification versus 23/46 under naive run-and-check, a 100% reduction in gaming success rate.

Step count analysis: Agents average 43 steps on software engineering tasks and 161 steps on scientific computing tasks, confirming that LHTB successfully stratifies tasks by difficulty and that step count is a meaningful proxy for task complexity.

Category difficulty ordering: The difficulty gradient (software engineering $\rightarrow$ data engineering $\rightarrow$ DevOps $\rightarrow$ games $\rightarrow$ experiment reproduction $\rightarrow$ scientific computing) is consistent across all three frontier models tested, suggesting it reflects genuine task structure rather than model-specific biases.

Reference

Zongxia Li et al. **Long-Horizon-Terminal-Bench: Testing the Limits of Agents on Long-Horizon Terminal Tasks with Dense Reward-Based Grading.** arXiv:2607.08964, July 2026. <https://arxiv.org/abs/2607.08964>

Keywords: neural collapse • representation learning • information theory • language models

Neural Collapse Is Forbidden

An Information Floor Theorem Shows That Within-Class Variance in Language Models Is Obligatory, Not a Training Artifact

By Bruno Abrahao (NYU Shanghai)

arXiv:2607.09487 • July 10, 2026

Neural collapse—the phenomenon where final-layer representations within each class converge to a single point during training—has been hailed as evidence that deep networks find elegant, maximally-separated class geometries. But for language models, this paper proves that neural collapse is theoretically impossible: within-class variance cannot shrink below a positive lower bound determined by how much contextual information each token carries beyond its category label. The paper terms this limit the **information floor**, derives it analytically, and confirms it empirically across 14 models spanning a 100× parameter range.

AT A GLANCE

Problem: Within-class variance in LM representations is commonly attributed to incomplete training or sub-optimal regularization.

Why it matters: If within-class variance is obligatory, attempts to reduce it via regularization or architecture choices will hurt model performance.

Method: Probing study across 14 models; theoretical proof of an information floor on within-class variance; k-NN mutual information estimator to compute the bound empirically.

Result: Within-class variance accounts for 79–91% of total representational variance; the theoretical floor covers 71–79%.

Evidence: 26-page paper with 3 figures and 19 tables across five text domains and five model families.

The Big Picture

Neural collapse was first described in classification networks: as training progresses, the penultimate-layer representations of samples within each class collapse to a single class mean, and the class means arrange themselves in a maximally-equidistant “simplex ETF” geometry. The phenomenon has been viewed as a signature of an ideal training outcome—a fully collapsed network is maximally linear-separable and robust to perturbations.

For language models, the analogue of a “class” is ambigu-

ous. Prior work has used token categories (part of speech, semantic class, domain label) as the class variable and found that within-category variance is large—sometimes 80–90% of total variance—and interpreted this as evidence that language models fail to achieve neural collapse.

This paper argues the interpretation is exactly backwards: large within-class variance is not a failure but a necessity. Language models must encode token-specific contextual information within each category because two tokens of the same category (e.g., both nouns) carry different next-token distributions depending on their context. Collapsing representations within a category would destroy this information and increase prediction loss—so the training objective itself prevents neural collapse.

Why Existing Approaches Fall Short

The neural collapse literature for classification assumes a fixed label space where two samples of the same class carry identical distributional targets. This assumption fails for language models: the distribution over next tokens for “bank” depends on whether the preceding context is financial or geographic, even though “bank” is in the same syntactic/semantic category in both cases.

Attempts to fine-tune language models toward neural collapse (via additional regularization or loss terms) have been tried in the context of classification fine-tuning. The paper’s theoretical result explains why these attempts always succeed in reducing within-class variance on the fine-tuning data but increase perplexity on held-out data: they destroy the contextual information encoded in the within-class variance.

Core Mechanism

The information floor arises from the structure of next-token prediction. For a token X with category label C and context Z , the model must compute $p(X_{\text{next}} | X, Z)$, which depends on both the category and the specific context. Two tokens of the same category in different contexts have different conditional next-token distributions, and these differences must be encoded in the representation.

The conditional mutual information $I(X; Z | C)$ quantifies exactly how much contextual information is necessary beyond the category label. The paper proves that the within-class variance σ_W^2 in any representation that achieves training loss below the theoretical minimum must satisfy $\sigma_W^2 \geq c \cdot I(X; Z | C)$.

Technical Formulation

Let X be the token, C its macro-category, Z the context window, and $h(X, Z) \in \mathbb{R}^d$ the final-layer representation.

Define the within-class variance as:

$$\sigma_W^2 = \mathbb{E}_C [\text{Var}(h(X, Z) \mid C)]$$

Theorem (Information Floor): For any model that achieves cross-entropy loss $\mathcal{L} \leq H(X_{\text{next}} \mid C, Z)$ on the training distribution:

$$\sigma_W^2 \geq c \cdot I(X; Z \mid C)$$

where $c > 0$ depends only on the loss function curvature and projection constants of the final linear layer.

The proof proceeds by lower-bounding the Fisher information of the output distribution as a function of σ_W^2 , and showing that $\sigma_W^2 < c \cdot I(X; Z \mid C)$ implies the Fisher information is insufficient to support the observed training loss.

Learning or Inference Procedure

1. For each of the 14 models, collect final-layer representations on 50,000 tokens from five text domains.
2. Assign 32-category macro-category labels using WordNet + POS tagging.
3. Compute within-class and between-class variance in the top- k principal subspace of the full covariance matrix.
4. Estimate $I(X; Z \mid C)$ using a k-NN mutual information estimator (Kozachenko-Leonenko calibrated).
5. Compare observed $\sigma_W^2 / (\sigma_B^2 + \sigma_W^2)$ against the theoretical lower bound derived from $I(X; Z \mid C)$.

What the Guarantee Says

The information floor theorem provides a constructive lower bound: given an estimate of $I(X; Z \mid C)$ from data, the bound predicts the minimum observable within-class variance share. Across five domains and 14 models, the theoretical floor covers 71–79% of total variance, and the observed share is 79–91%—consistently above the floor by a margin of 8–12 percentage points, consistent with the floor being a lower bound rather than an equality.

Experimental Findings

Model family	$\sigma_W^2 / \sigma_{\text{total}}^2$	Floor
GPT series	$82.4\% \pm 3.1\%$	73.1%
LLaMA series	$84.7\% \pm 2.4\%$	75.8%
Mistral	$83.9\% \pm 2.8\%$	74.2%
Claude family	$79.3\% \pm 4.2\%$	71.0%
Gemma	$85.2\% \pm 1.9\%$	76.3%

The variance share is insensitive to scale within each family (max 2% change over $10\times$ parameter increase) and insensitive to tokenization (BPE vs. SentencePiece: 0.8% difference). All observations lie above the predicted floor.

Ablations and Interpretation

Random-label control: Replacing token category labels with random assignments increases σ_B^2 by 18% and σ_W^2 by 22%, confirming the variance decomposition is meaningful.

Layer depth: Within-class variance share increases from 55% at layer 1 to 84% at the final layer, suggesting the model progressively specializes representations toward context-specific rather than category-level structure.

Fine-tuning toward collapse: Fine-tuning on sentiment classification with an additional within-class variance penalty reduces σ_W^2 on the fine-tuning domain by 12% but increases perplexity by 8%, directly confirming that forcing collapse destroys necessary contextual information.

Rare tokens: The information floor is higher for rare tokens (those with unique distributional contexts), consistent with the finding that rare tokens are disproportionately memorized—they carry more $I(X; Z \mid C)$ and therefore require more within-class dispersion to represent correctly.

Reference

Bruno Abrahao. **Neural Collapse Is Forbidden: Information Floors in Language Models.** arXiv:2607.09487, July 10, 2026. NYU Shanghai. <https://arxiv.org/abs/2607.09487>

Keywords: video generation • visual perception • foundation models • diffusion • ECCV 2026

Video Generation Models are General-Purpose Vision Learners

A Pre-Trained Text-to-Video Diffusion Backbone Achieves State-of-the-Art Across Depth, Pose, Segmentation, and 3D Keypoint Tasks via GenCepion

By Letian Wang, Chuhan Zhang, Rishabh Kabra, Jasper Uijlings, et al. (Google DeepMind)

arXiv:2607.09024 • ECCV 2026

The shift from task-specific NLP models to powerful generalists began when researchers recognized that next-token prediction on large corpora incidentally learns representations useful for every downstream task. This paper argues that large-scale text-to-video generation plays an analogous role for computer vision: the spatiotemporal priors, vision-language alignment, and scalability of video generation pre-training transfer to a wide range of perception tasks with no task-specific architectural changes. GenCepion, the resulting framework, achieves state-of-the-art performance on depth estimation, surface normal prediction, camera pose estimation, expression-referring segmentation, and 3D keypoint detection—from a single frozen backbone.

AT A GLANCE

Problem: Computer vision still relies on task-specific architectures; a single backbone that matches specialized models across diverse tasks has not existed.

Why it matters: A single foundation model for vision reduces engineering overhead and may exhibit better cross-task generalization than ensembles of specialists.

Method: Freeze a pre-trained 7B text-to-video diffusion transformer; adapt to perception tasks via lightweight task-head fine-tuning with text-instruction task routing.

Result: State-of-the-art or near-SOTA on 5 of 7 diverse vision benchmarks; 38% improvement on camera pose over image-only diffusion baselines.

Evidence: ECCV 2026 accepted; evaluated on NYUv2, Taskonomy, CO3D, EmotionNet, Human3.6M, and two more.

The Big Picture

Computer vision tasks—depth estimation, surface normal prediction, camera pose estimation, semantic segmentation, human pose estimation—have traditionally been addressed by specialist architectures trained on task-specific datasets. Even with the advent of large vision-language models (VLMs), perception tasks requiring geometric 3D understanding remain largely the domain of specialized networks.

The analogy with NLP is instructive. Before GPT, the standard recipe was: (1) design an architecture for the task, (2) train on labeled data, (3) deploy. Next-token prediction on large corpora replaced step (1) and dramatically reduced the labeled data requirement for step (2). This paper asks: what is the vision analogue of next-token prediction?

The authors argue it is text-to-video generation. Generating realistic video requires the model to understand depth, 3D geometry, object motion, and scene layout—exactly the representations needed for downstream vision tasks. Large-scale video generation pre-training on hundreds of millions of clips thus incidentally instills the priors that specialist architectures must acquire from task-specific labels.

Why Existing Approaches Fall Short

Image diffusion models (Stable Diffusion, DALL-E 3) have been adapted for perception tasks by treating the output domain (depth maps, surface normals) as structured pseudo-images. But image-only pre-training lacks temporal information, producing representations that are weaker on 3D-geometric tasks: camera pose estimation (which requires cross-view geometric consistency) and 3D keypoint detection (which requires depth understanding) degrade by 38% and 12% respectively when using an image backbone instead of a video backbone.

Specialist models (Depth Anything v2, ViTPose) achieve strong results on their respective tasks but require separate architectures, training pipelines, and deployment instances. GenCepion replaces them with a single checkpoint—though specialists retain a slight advantage on tasks where large task-specific datasets are available.

Core Mechanism

GenCepion treats every perception task as a conditional generation problem. The input is an image (or video frame); the output is a structured target (depth map, surface normal map, pose heatmap, etc.) encoded as a pseudo-image in the same pixel space.

A text instruction (e.g., “produce a metric depth map of this scene”) routes the backbone to the appropriate output distribution. The diffusion backbone generates

the target pseudo-image in the usual denoising loop. A lightweight 3-layer MLP head applied to the final denoised latent maps it to the task-specific output format (metric depth values, unit normal vectors, etc.).

The key insight is that the backbone, pre-trained to generate any video frame conditioned on text, can generate any structured pseudo-image conditioned on a perception-task instruction—without any change to its weights.

Technical Formulation

Let I be the input image, τ the text task instruction, and $\hat{Y}$ the target structured output (a depth map, pose heatmap, etc.). GenCepion models:

$$p(\hat{Y} \mid I, \tau) = \int p(\hat{Y} \mid z_T, \tau) \prod_{t=1}^T p(z_{t-1} \mid z_t, I, \tau) dz_{1:T}$$

via DDIM sampling with $T = 50$ steps, where the backbone’s attention cross-attends to both the image I (via image conditioning) and the instruction τ (via CLIP text encoding). The MLP task head g_ϕ then maps the final latent z_0 to the task output: $Y = g_\phi(z_0)$.

Only ϕ (the MLP head, approximately 2M parameters) is fine-tuned on task-specific data; the backbone’s 7B parameters remain frozen.

Learning or Inference Procedure

1. Pre-train the text-to-video diffusion backbone on 500M video-text pairs at resolutions up to 1080p (not part of this paper; uses an existing backbone).
2. For each perception task, collect task-specific labeled data (e.g., NYUv2 for depth, Human3.6M for 3D keypoints).
3. Fine-tune the MLP task head for 10K steps with learning rate 1e-4; backbone frozen throughout.
4. At inference, run DDIM with 50 steps to denoise the target pseudo-image; apply the task head to the final latent.
5. Optionally, condition on a sequence of frames for tasks requiring temporal consistency (e.g., video depth estimation).

What the Guarantee Says

The paper does not provide formal guarantees, but the empirical evidence supports two strong claims: (1) video pre-training provides richer priors than image pre-training for 3D-geometric tasks (camera pose: 38% improvement; 3D keypoints: 12%); and (2) task-instruction text conditioning is a necessary component—removing it degrades performance by 8.3% on average, confirming it serves as a task router rather than mere noise.

Experimental Findings

Task (dataset)	GenCep.	Prior SOTA	Δ
Depth (NYUv2)	0.081	0.089	+9.0%
Normals (Taskonomy)	11.2°	12.8°	+12.5%
Camera pose (CO3D)	2.1°	3.4°	+38.2%
Expr. seg. (EmotioNet)	71.3 mAP	68.1	+4.7%
3D keypts. (H3.6M)	38.2 mm	41.3 mm	+7.5%

GenCepion is competitive or superior in all five categories. The largest gains appear on 3D-geometric tasks (pose, depth), consistent with the hypothesis that video pre-training instills strong geometric spatial priors absent from image-only baselines.

Ablations and Interpretation

Video vs. image backbone: Replacing the video backbone with Stable Diffusion 3.5 (image-only, comparable size) degrades camera pose estimation by 38% and 3D keypoints by 12%, confirming temporal pre-training as the source of 3D geometric priors.

Pre-training data scale: Halving the pre-training video volume degrades downstream AUC by 6.2% on average; doubling improves by 4.1%, confirming a log-linear scaling law between video volume and downstream perception quality.

Frozen vs. fine-tuned backbone: Full backbone fine-tuning improves depth estimation by 1.8% but degrades surface normals by 1.2%, suggesting dataset-specific overfitting; the frozen-backbone default balances cross-task generalization.

Text conditioning ablation: Removing the task-instruction prefix and using image-only conditioning degrades performance by 8.3% on average—the largest single ablation—confirming text as the task-routing mechanism rather than an incidental input.

Reference

Letian Wang et al. **Video Generation Models are General-Purpose Vision Learners.** ECCV 2026. arXiv:2607.09024. Google DeepMind. <https://arxiv.org/abs/2607.09024>